# Multi-Node Private Cloud Infrastructure Using OpenStack and Kolla-Ansible

**Project period:** March 2026 – September 2026


## Abstract

Designed, deployed and expanded a multi-node private cloud environment using OpenStack Antelope (2023.1) and Kolla-Ansible on Ubuntu Server 22.04. The platform provides shared compute, networking and storage resources for student users, together with cloud services for file storage, monitoring, database access and Kubernetes. The infrastructure began with Controller, Compute1 and Storage nodes and was expanded with a Dell Precision workstation acting as Compute2.


## Project Objectives

- Build a multi-node OpenStack environment using dedicated physical nodes.
- Provide isolated cloud resources and services for multiple student users.
- Expand compute capacity by integrating an additional high-performance workstation.
- Expose selected cloud services remotely through Cloudflare Tunnel and a custom domain.
- Provide storage, monitoring, database and Kubernetes services for student workloads.

## Infrastructure Architecture

The core OpenStack environment is deployed across four physical nodes:

- Controller: Hosts OpenStack control-plane and API services.
- Compute1: Provides virtual machine compute capacity.
- Storage: Provides storage resources for the cloud environment.
- Compute2: Dell Precision 7960 workstation integrated to increase compute and storage capacity.

## Compute2 Hardware Expansion

The Dell Precision 7960 workstation was provided by Samsung Innovation Lab for the cloud project. Its reported configuration includes an Intel Xeon w9-3475X processor with 36 cores and 72 threads, 256 GB RAM and approximately 13 TB of raw storage capacity. It was integrated as Compute2, extending the original three-node deployment.

## OpenStack Platform and Networking

OpenStack Antelope (2023.1) was deployed using Kolla-Ansible, with services running in Docker containers.

- Nova: Virtual machine provisioning and lifecycle management.
- Neutron: Tenant networking, virtual routers and network connectivity.
- Keystone: Identity, authentication and project-based access.
- Glance: Virtual machine image management.
- Cinder and Swift: Block storage and object storage.
- Skyline and Horizon: User-facing and administrative cloud dashboards.
- Heat: Template-based infrastructure provisioning, tested as an Infrastructure-as-Code approach.

Networking uses Open vSwitch, VXLAN tenant networks, provider networking, routing and floating IP connectivity. The environment is configured to separate tenant workloads and provide controlled access to cloud resources.

## Cloud Services for Student Users

Selected services are made available to classmates through dedicated web endpoints under openstacklab.me.

### Cloud Dashboard – cloud.openstacklab.me

Configured OpenStack Skyline as the student-facing dashboard. Horizon administrative access is restricted to the cloud providers.

### Object Storage – drive.openstacklab.me

Integrated OpenStack Swift with Nextcloud to provide a familiar file-management interface. Students can access their object storage through the web interface and configured Android and iOS mobile applications, with user data separated through account-level access and permissions.

### Monitoring – monitor.openstacklab.me

Implemented Prometheus and Grafana monitoring. User-facing dashboards are scoped to relevant projects and virtual machines, with 24 panels selected for the student view. A more extensive administrative Grafana environment is maintained for cloud operators.

### Database Service – db.openstacklab.me

Configured relational database service access so users can obtain database environments for their projects.

### Kubernetes as a Service – kubernetes.openstacklab.me

Extended the platform by making a Kubernetes environment available as a service. The cluster is deployed on four OpenStack-provisioned virtual machines: one control-plane node and three worker nodes. Each VM is configured with 24 GB RAM and 6 vCPUs. Kubernetes v1.30.14 was deployed using kubeadm, with containerd as the runtime and Calico for pod networking.

A dedicated Kubernetes dashboard is hosted at https://kubernetes.openstacklab.me. Cloudflare Tunnel is configured to provide remote access to the Kubernetes API endpoint. Users can connect from their own laptops using Windows PowerShell or Command Prompt with kubectl and an appropriately configured kubeconfig. Access must be granted using suitable authentication and role-based permissions.

The cluster is a separate workload hosted on OpenStack, not an OpenStack service. Its detailed architecture, deployment and workload testing are documented as a separate Kubernetes project.

## Remote Access and Service Exposure

Configured Cloudflare Tunnel to publish selected cloud services through openstacklab.me. Dedicated subdomains provide remote access to Skyline, Nextcloud, Grafana, database services and the Kubernetes dashboard. The Kubernetes API is also made remotely reachable through the configured tunnel for authenticated client access.

## Additional Developer Services

- Deployed Jupyter as a service with Keycloak-based authentication.
- Deployed Coder as a browser-based development environment within Kubernetes.

## Resilience and High-Availability Considerations

The platform was designed with logical resilience and service-continuity principles, including tenant separation, workload replication and recovery testing in Kubernetes. A fully redundant OpenStack control plane was not implemented because available hardware resources were limited and were prioritized for the deployed infrastructure and services. The environment should therefore be described as a multi-node private cloud with HA-oriented design considerations, not as a fully redundant OpenStack control plane.

## Implementation Challenges

The project involved troubleshooting across deployment, networking, storage and service-integration layers. Work included resolving Kolla-Ansible and Ansible dependency issues, configuring Open vSwitch and tenant networking, integrating additional compute hardware, exposing selected services through Cloudflare Tunnel, and configuring user-scoped access to shared services.

## Results and Outcomes

- Expanded the original three-node OpenStack environment into a four-node physical infrastructure by adding Compute2.
- Established a multi-tenant private cloud environment for student use.
- Provided a Skyline-based cloud dashboard while restricting administrative access.
- Made Swift-backed file storage accessible through Nextcloud web and mobile applications.
- Provided project- and VM-scoped monitoring dashboards through Grafana.
- Configured relational database access and tested Heat-based infrastructure provisioning.
- Provided Kubernetes access as a service, including a dedicated dashboard and remote API connectivity.
- Deployed Jupyter and Coder as additional developer services.

## Technologies Used

**Cloud:** OpenStack Antelope (2023.1), Kolla-Ansible, Nova, Neutron, Keystone, Glance, Cinder, Swift, Skyline, Horizon, Heat.

**Infrastructure:** Ubuntu Server 22.04, Linux, Docker, Ansible, Dell Precision 7960, Open vSwitch, VXLAN.

**Storage and Monitoring:** Nextcloud, Prometheus, Grafana.

**Container Platform:** Kubernetes v1.30.14, kubeadm, containerd, Calico, kubectl, Coder.

**Developer and Identity Services:** Jupyter, Keycloak.

**Remote Access:** Cloudflare Tunnel and custom DNS subdomains.

## Conclusion

This project demonstrates the design, deployment and expansion of a multi-node private cloud using OpenStack. By integrating additional physical compute capacity and extending the platform with storage, monitoring, database and Kubernetes services, the environment evolved into a shared cloud platform for student workloads. The work provided practical experience in cloud infrastructure, Linux administration, software-defined networking, storage integration, multi-tenancy, remote service access and platform operations.

### Project Scope Note

Kubernetes is deployed as a separate workload on OpenStack-provisioned virtual machines and is also offered through a dedicated user-facing dashboard and remote API access. It is intentionally documented separately as its own project. High-availability wording refers to design considerations and workload-level resilience, not a fully redundant OpenStack control plane.